

PCS-302 : Data Structures Lab

Deque (Double-Ended Queue) Using an Array

Problem Statement

Design an algorithm and implement a deque (double-ended queue) using a program. A double-ended queue is a queue in which insertion and deletion can be performed at both ends. The program should implement the following operations: insertFront(), insertEnd(), deleteFront(), deleteEnd(), isEmpty(), isFull(), printFront() and printEnd().

Example

```
insertEnd(10)    --> Deque: 10
insertEnd(20)    --> Deque: 10 20
insertFront(5)   --> Deque: 5 10 20
insertEnd(30)    --> Deque: 5 10 20 30

deleteFront()    --> removes 5, Deque: 10 20 30
deleteEnd()      --> removes 30, Deque: 10 20

printFront():    10 20    (front to end)
printEnd():      20 10    (end to front)
```

Algorithm (Array-Based, Using Shifting)

```
Create():
    count = 0

InsertFront(k):
    if count == MAX: print "Overflow", return
    for i = count down to 1:
        arr[i] = arr[i-1]           // shift everything right
    arr[0] = k
    count = count + 1

InsertEnd(k):
    if count == MAX: print "Overflow", return
    arr[count] = k
    count = count + 1

DeleteFront():
    if count == 0: print "Underflow", return
    for i = 0 to count-2:
        arr[i] = arr[i+1]           // shift everything left
    count = count - 1

DeleteEnd():
    if count == 0: print "Underflow", return
    count = count - 1

IsEmpty():
    return (count == 0)

IsFull():
```

```
    return (count == MAX)
```

```
PrintFront():  
    for i = 0 to count-1: print arr[i]
```

```
PrintEnd():  
    for i = count-1 down to 0: print arr[i]
```

C Program

```
#include <stdio.h>  
#define MAX 5  
  
int arr[MAX];  
int count = 0;  
  
int isEmpty() {  
    if (count == 0)  
        return 1;  
    else  
        return 0;  
}  
  
int isFull() {  
    if (count == MAX)  
        return 1;  
    else  
        return 0;  
}  
  
void insertFront(int k) {  
    int i;  
    if (isFull() == 1) {  
        printf("Deque Overflow. Cannot insert %d at front.\n", k);  
        return;  
    }  
    // shift all elements one position to the right  
    for (i = count; i > 0; i--) {  
        arr[i] = arr[i - 1];  
    }  
    arr[0] = k;  
    count = count + 1;  
    printf("%d inserted at front.\n", k);  
}  
  
void insertEnd(int k) {  
    if (isFull() == 1) {  
        printf("Deque Overflow. Cannot insert %d at end.\n", k);  
        return;  
    }  
    arr[count] = k;  
    count = count + 1;  
    printf("%d inserted at end.\n", k);  
}  
  
void deleteFront() {
```

```

    int i;
    if (isEmpty() == 1) {
        printf("Deque Underflow. Cannot delete from front.\n");
        return;
    }
    printf("%d deleted from front.\n", arr[0]);
    // shift all elements one position to the left
    for (i = 0; i < count - 1; i++) {
        arr[i] = arr[i + 1];
    }
    count = count - 1;
}

void deleteEnd() {
    if (isEmpty() == 1) {
        printf("Deque Underflow. Cannot delete from end.\n");
        return;
    }
    printf("%d deleted from end.\n", arr[count - 1]);
    count = count - 1;
}

void printFront() {
    int i;
    if (isEmpty() == 1) {
        printf("Deque is empty.\n");
        return;
    }
    printf("Deque (front to end): ");
    for (i = 0; i < count; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void printEnd() {
    int i;
    if (isEmpty() == 1) {
        printf("Deque is empty.\n");
        return;
    }
    printf("Deque (end to front): ");
    for (i = count - 1; i >= 0; i--) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    insertEnd(10);
    insertEnd(20);
    insertFront(5);
    insertEnd(30);

    printFront();

```

```
    printEnd();

    deleteFront();
    deleteEnd();

    printFront();

    if (isEmpty() == 1)
        printf("Deque is empty.\n");
    else
        printf("Deque is not empty.\n");

    return 0;
}
```